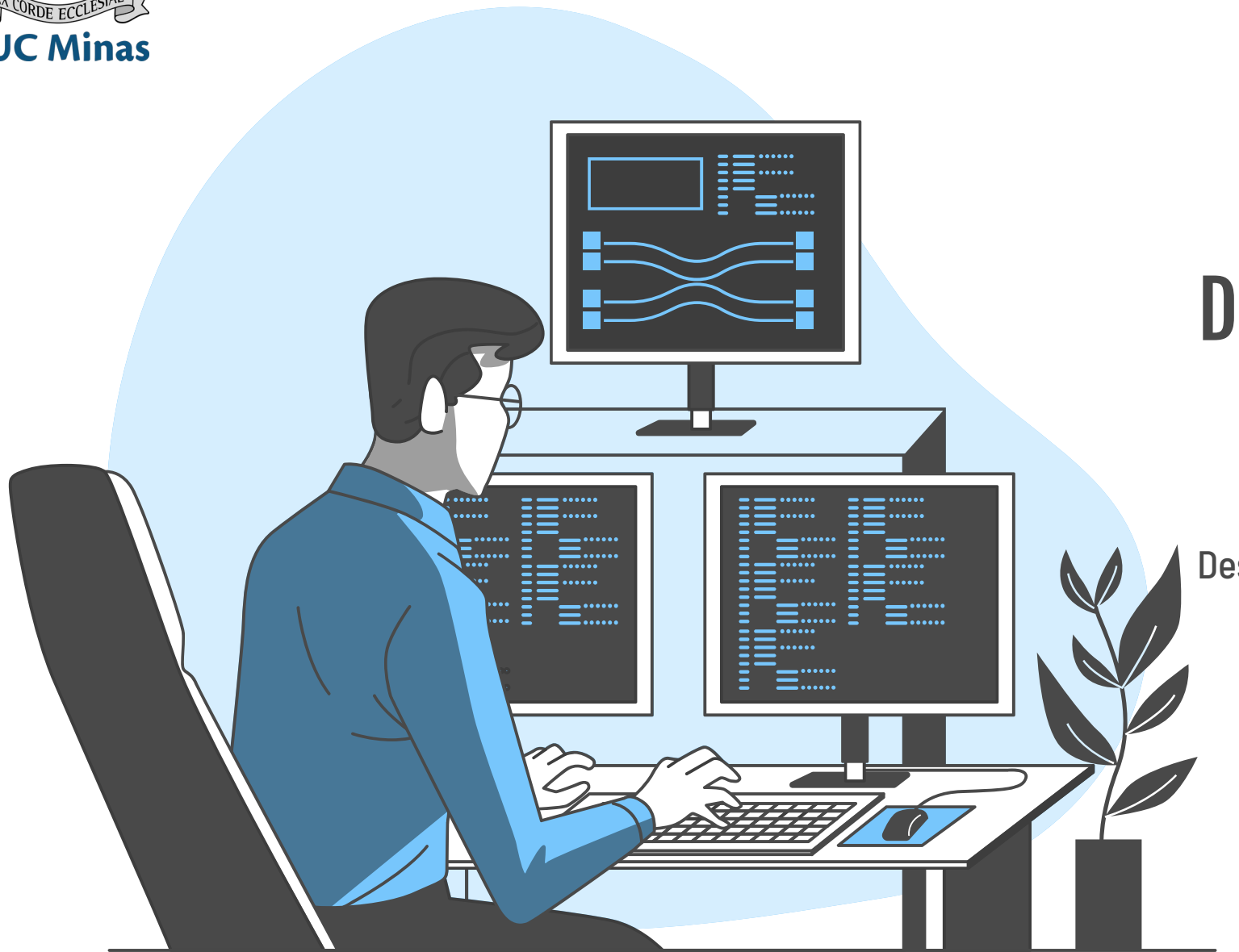
PUC Minas



Aula 04 · Deploy

Dockerfile do front-end e docker compose

Desenvolvimento e Integração de Aplicações Web

Prof. Dr. João Paulo Aramuni

Engenharia de Software · PUC Minas

github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web

Apresentação da aula

Nesta aula você vai

1

Escrever o Dockerfile do front: **Node** para o build e **nginx** para servir

2

Passar `VITE_API_URL` no build com `ARG`

3

Configurar o `nginx.conf` para SPA (`try_files`)

4

Escrever um `docker-compose.yml` com `mongo`, `back` e `front`

5

Usar `docker compose up --build`, `logs` e `down`

Antes de começar

- Aula 03 desta pasta
- Front-end, aula 07 (variáveis VITE_)

Duração sugerida

100 min

Código da disciplina

Projetos de referência na pasta PROJETOS do GitHub

Roteiro

01 Quando o front precisa de Docker

02 Dockerfile do front

03 docker compose: tudo de uma vez

04 Aprofundando: compose para o desenvolvimento

01

Quando o front precisa de Docker

Quando o front precisa de Docker

No Vercel **não precisamos** de Dockerfile para o front: ele faz o build sozinho. O Dockerfile do front serve para rodar o **projeto inteiro localmente** com docker compose (sem instalar Java, Node nem MongoDB) ou para publicar em serviços que só aceitam containers. Os projetos `PalpiteDTI` e `DronesDTI` têm os dois.

01

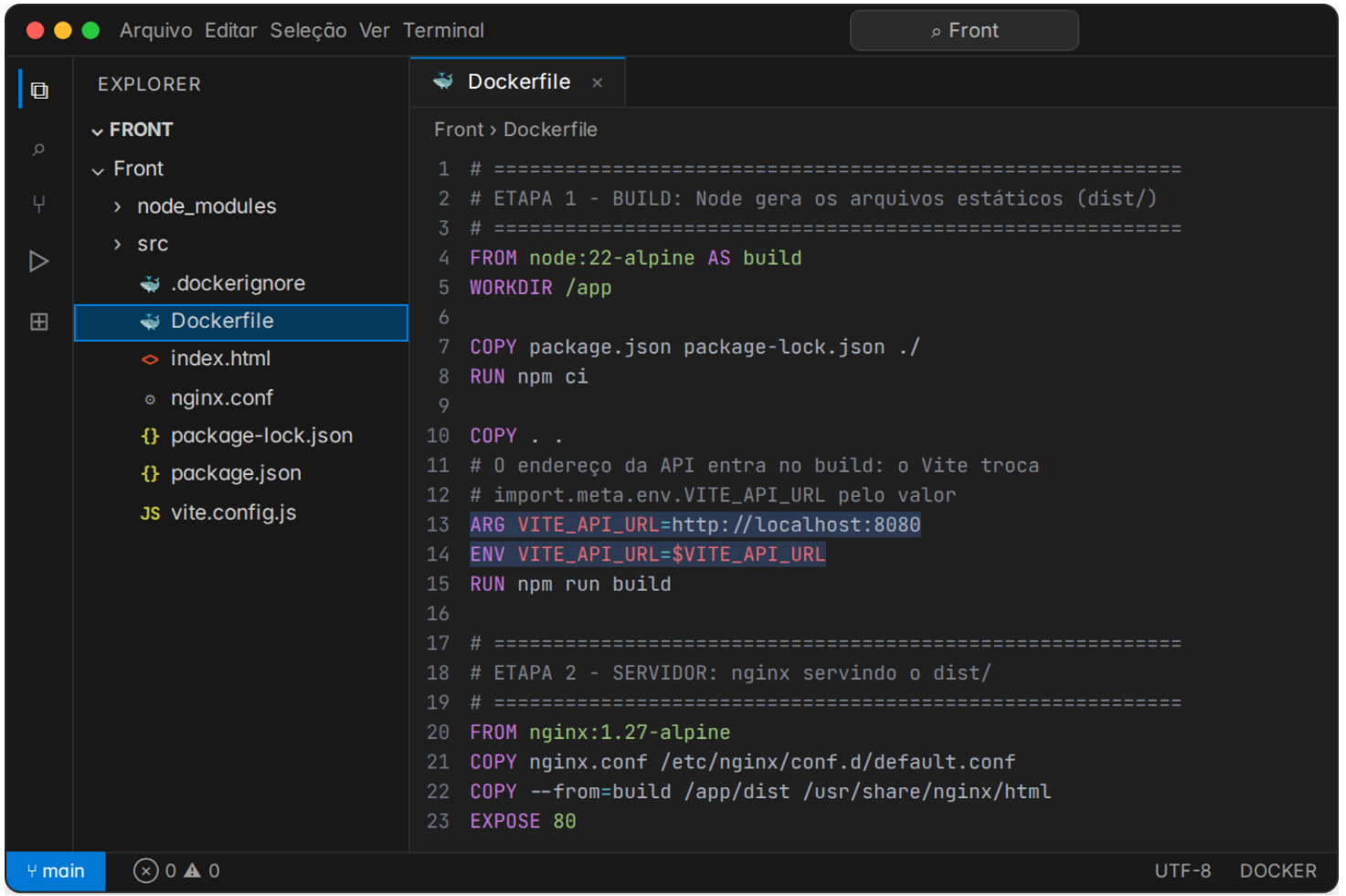
Quando o front precisa de Docker

Aula 04 · Deploy

02

Dockerfile do front

Dockerfile do front



Dockerfile do front do `PalpiteDTI`: Node só para o build, nginx para servir.

Dockerfile do front

- `npm ci` instala exatamente o `package-lock.json` (mais rápido e previsível que `npm install` em builds automáticos).
- `ARG` é um valor disponível **só durante o build**, passado com `--build-arg`. Como o Vite grava a URL dentro do JavaScript no build, ela precisa ser `ARG`, e não uma variável de execução (`-e`). Mudou a URL? Reconstrua a imagem.
- **nginx** é um servidor web leve, ótimo para arquivos estáticos.

02

Dockerfile do front

Aula 04 · Deploy

Dockerfile do front

Front/nginx.conf

```
server {
    listen 80;
    root /usr/share/nginx/html;
    index index.html;

    # SPA: qualquer rota (/ranking, /admin)
    # devolve o index.html e o React Router
    # decide o que mostrar
    location / {
        try_files $uri $uri/ /index.html;
    }

    # arquivos com hash no nome: cache longo
    location /assets/ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }
}
```

Front/.dockerignore

```
node_modules
dist
.env
.env.local
.git
```

Observação

O `try_files` faz no nginx o mesmo papel do `vercel.json` no Vercel e do `SpaConfig` no Spring (FullStack, aula 07): F5 em `/ranking` não dá 404.

Dockerfile do front

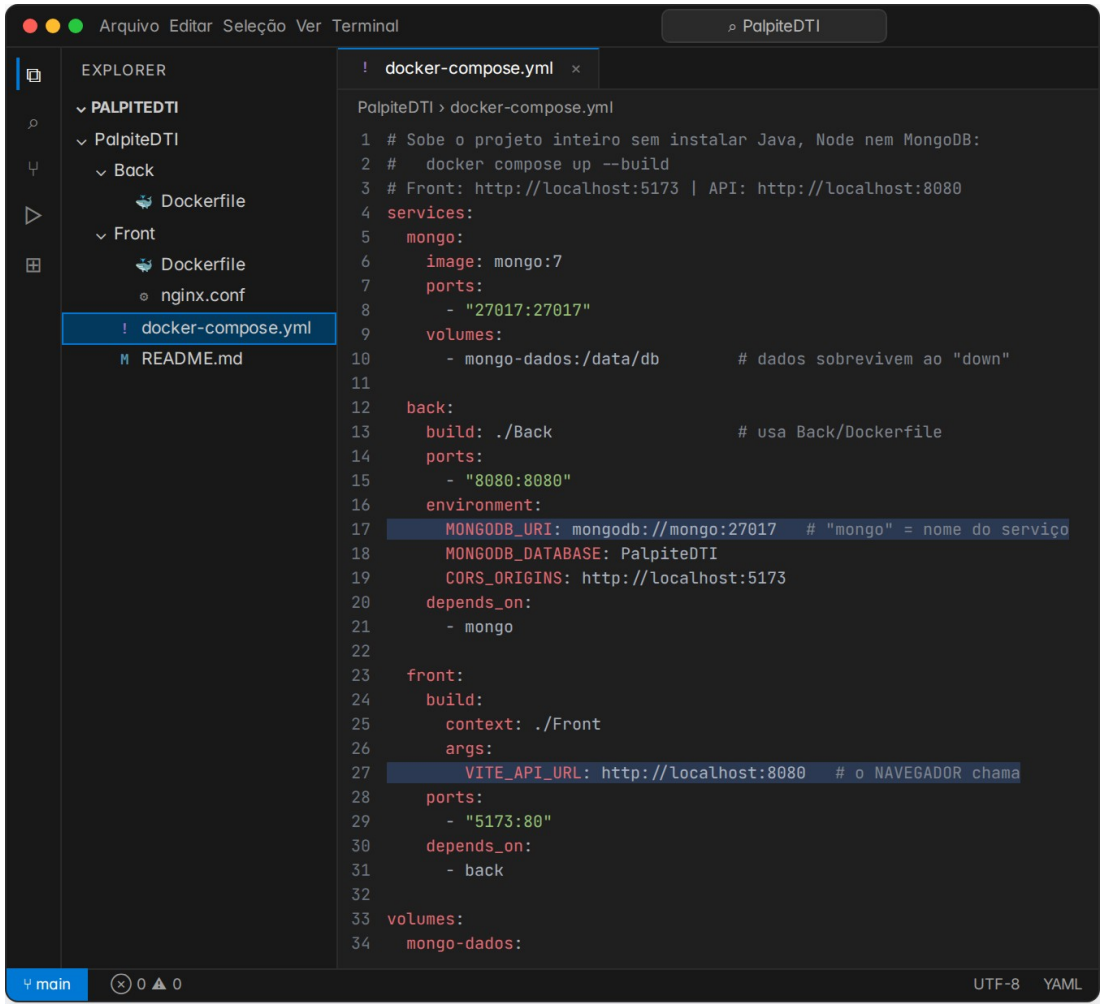
Construindo e rodando o front

```
docker build -t meu-front --build-arg VITE_API_URL=http://localhost:8080 .
docker run --rm -p 5173:80 meu-front      # abra http://localhost:5173
```

03

docker compose: tudo de uma vez

docker compose: tudo de uma vez



Rodar três `docker run` com redes e variáveis é trabalhoso. O **docker compose** descreve todos os serviços em um arquivo YAML na raiz do repositório:

`docker-compose.yml` do `PalpiteDTI`.

docker compose: tudo de uma vez

Atenção

Duas URLs diferentes, e é proposital. O **back** fala com o banco por dentro da rede do compose, então usa o nome do serviço: `mongodb://mongo:27017`. O **front** roda no **navegador**, fora do Docker, então precisa de `http://localhost:8080`. Colocar `http://back:8080` no `VITE_API_URL` é um erro clássico: o navegador não conhece o nome `back`.

docker compose: tudo de uma vez

Terminal

```
aluno@pc:~/PalpiteDTI$ docker compose up --build
[+] Building 98.1s (29/29) FINISHED
[+] Running 4/4
 ✔ Network palpitedti_default    Created
 ✔ Container palpitedti-mongo-1  Started
 ✔ Container palpitedti-back-1   Started
 ✔ Container palpitedti-front-1  Started
back-1   | Tomcat started on port 8080 (http)
back-1   | Started PalpiteDtiApplication in 5.2 seconds
```

Um comando sobe banco, API e site. `Ctrl+C` para; `docker compose down` remove os containers.

docker compose: tudo de uma vez

Comando	O que faz
`docker compose up --build`	Constrói as imagens e sobe tudo (mostrando os logs)
`docker compose up -d`	Sobe em segundo plano
`docker compose logs -f back`	Acompanha os logs de um serviço
`docker compose ps`	Lista os serviços
`docker compose down`	Para e remove os containers (o volume fica)
`docker compose down -v`	Remove também os volumes (apaga os dados do banco)

docker compose: tudo de uma vez

✓ **Dica**

Para usar o **MongoDB Atlas** em vez do container local, remova o serviço `mongo` e troque o `MONGODB_URI` do `back` pela *connection string* do Atlas. Para não escrever a senha no YAML, use `MONGODB_URI: ${MONGODB_URI}` e um arquivo `.env` na raiz (fora do Git): o compose lê esse arquivo sozinho.

04

Aprofundando: compose para o desenvolvimento

Aprofundando: compose para o desenvolvimento

depends_on com healthcheck

```yaml
services:
  mongo:
    image: mongo:7
    healthcheck:
      test: ["CMD", "mongosh", "--quiet", "--eval", "db.adminCommand('ping')"]
      interval: 5s
      retries: 10

  back:
    build: ./Back
    env_file: .env                    # MONGODB_URI, JWT_SECRET... (fora do Git)
    depends_on:
      mongo:
        condition: service_healthy    # só sobe quando o Mongo responder
    restart: unless-stopped
```

Aprofundando: compose para o desenvolvimento

Comando	Uso
`docker compose up --build back`	Reconstrói e sobe só um serviço
`docker compose exec back sh`	Terminal dentro do serviço
`docker compose restart back`	Reinicia um serviço
`docker compose config`	Mostra o arquivo final, com variáveis resolvidas
`docker compose pull`	Atualiza as imagens prontas (mongo, nginx)

Front no nginx x front no Vercel

nginx em container

- Você controla cache, compressão e proxy
- Precisa de um servidor rodando
- Bom para o compose local e serviços só de containers

vs

Vercel

- Build e CDN gerenciados
- Previews por branch
- Padrão da disciplina para produção

Aprofundando: compose para o desenvolvimento

nginx.conf com compressão e cache

```
server {
    listen 80;
    root /usr/share/nginx/html;
    gzip on;
    gzip_types text/css application/javascript application/json image/svg+xml;

    location = /index.html {
        add_header Cache-Control "no-cache";      # sempre busca a versão nova
    }
    location /assets/ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }
    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

Variáveis no compose

environment

Valores escritos direto no YAML (só o que não é segredo).

env_file

Lê um arquivo `.env` (fora do Git) com os segredos.

${VAR}

Substitui pelo valor do `.env` da raiz ou do sistema.

args

Valores de build, como o `VITE_API_URL` do front.

Aprofundando: compose para o desenvolvimento

Erro comum

`mongo` não encontrado ou **`Connection refused` para `localhost:27017` dentro do back**: dentro do compose, `localhost` é o próprio container do back. Use o nome do serviço (`mongodb://mongo:27017`). Só o navegador, que está fora do Docker, usa `localhost`.

Aprofundando: compose para o desenvolvimento

1. Escreva os Dockerfiles do Back e do Front.
2. Crie o `docker-compose.yml` na raiz com `mongo`, `back` e `front`.
3. Crie o `.env` da raiz com os segredos e adicione ao `.gitignore`.
4. `docker compose up --build` e abra `http://localhost:5173`.
5. Documente o comando no README: qualquer pessoa sobe o projeto sem instalar Java nem Node.

05

Quiz de revisão

Quiz · questão 1 de 5

Pergunta

Por que `VITE_API_URL` é ARG e não variável de execução?

A Estética

B O Vite grava o valor no JavaScript durante o build

C O nginx exige

D Por causa do CORS

Quiz · questão 1 de 5

Resposta

Por que `VITE_API_URL` é ARG e não variável de execução?

A Estética

✓ O Vite grava o valor no JavaScript durante o build

C O nginx exige

D Por causa do CORS

Por quê: Mudou a URL? Reconstrua a imagem.

Pergunta

Qual papel o nginx tem no Dockerfile do front?

A Compilar o React

B Servir os arquivos do dist/

C Conectar ao banco

D Gerar o token

Resposta

Qual papel o nginx tem no Dockerfile do front?

A Compilar o React

✓ Servir os arquivos do dist/

C Conectar ao banco

D Gerar o token

Por quê: O Node só é usado no build.

Pergunta

No compose, como o back acessa o Mongo?

A localhost:27017

B mongodb://mongo:27017 (nome do serviço)

C IP do computador

D Pelo Atlas apenas

Resposta

No compose, como o back acessa o Mongo?

A localhost:27017

✓ mongodb://mongo:27017 (nome do serviço)

C IP do computador

D Pelo Atlas apenas

Por quê: Dentro do Docker, localhost é o próprio container.

Pergunta

Por que `VITE_API_URL: http://back:8080` não funciona?

A Porta errada

B Quem chama é o navegador, que não conhece o nome back

C Falta HTTPS

D Falta CORS

Resposta

Por que `VITE_API_URL: http://back:8080` não funciona?

A Porta errada

✓ Quem chama é o navegador, que não conhece o nome back

C Falta HTTPS

D Falta CORS

Por quê: Use http://localhost:8080.

Pergunta

O que `docker compose down -v` faz a mais que `down`?

A Nada

B Apaga também os volumes (dados do banco)

C Reconstrói as imagens

D Faz deploy

Resposta

O que `docker compose down -v` faz a mais que `down`?

A Nada

✓ Apaga também os volumes (dados do banco)

C Reconstrói as imagens

D Faz deploy

Por quê: Use com cuidado.

Resumo da aula

- ✓ Front em Docker: `node:22-alpine` faz `npm ci` + `npm run build`; `nginx` serve o `dist/`.
- ✓ `VITE_API_URL` entra como `ARG` no build, porque o Vite grava o valor no JavaScript.
- ✓ `nginx.conf` com `try_files ... /index.html` para rotas de SPA.
- ✓ `docker compose up --build` sobe banco, back e front; serviços se enxergam pelo nome.
- ✓ Back usa `mongo:27017` (rede interna); front usa `localhost:8080` (navegador).

Exercícios

1. Crie Dockerfile e `nginx.conf` para o seu front e rode com `docker run`.

2. Escreva o `docker-compose.yml` do seu projeto e suba tudo com um comando.

3. Troque o `mongo` local pelo Atlas usando um `.env` na raiz.

4. Explique por que `VITE_API_URL: http://back:8080` não funciona.

Para ir além

- docs.docker.com/compose
- hub.docker.com/_/nginx
- Projetos `FullStack/PalpiteDTI` e `FullStack/DronesDTI`

Repositório da disciplina

Todos os projetos citados estão na pasta PROJETOS

Abrir no GitHub

Links e versões citados podem ter mudado. Confira sempre a documentação oficial.



Obrigado!

Dúvidas sobre a aula 04?

Prof. Dr. João Paulo Aramuni
Engenharia de Software · PUC Minas
Desenvolvimento e Integração de Aplicações Web

GitHub da disciplina
https://github.com/joaopauloaramuni/desenvolvimento-e-integracao-de-aplicacoes-web